

Pearls AQI Predictor

End-to-End Machine Learning Pipeline for Air Quality Forecasting
Karachi, Pakistan · 3-Day AQI Forecast · Real-Time Dashboard

Submitted By	Rana Mohsin
Date	June 6, 2026
Organization	10Pearls
Domain	Data Sciences
University	Iqra University, Karachi
CGPA	3.81
GitHub	github.com/ranamohsincodes/AQI-PIPELINE

Technology Stack

Python 3.10	Scikit-learn	TensorFlow/Keras	Flask REST API
Streamlit	GitHub Actions	OpenWeatherMap API	SHAP

Table of Contents

1.	Executive Summary
2.	Project Overview & Objectives
3.	System Architecture
4.	Data Collection & Feature Pipeline
5.	Feature Engineering
6.	Model Training & Evaluation
7.	CI/CD Automation — GitHub Actions
8.	Web Dashboard & Flask API
9.	SHAP Explainability
10.	EDA Findings
11.	Deliverables Checklist
12.	Honest Limitations & What Was Not Implemented
13.	Conclusion

1. Executive Summary

This report documents the complete development of an end-to-end Air Quality Index (AQI) prediction system for Karachi, Pakistan, built as the capstone project for the 10Pearls Data Sciences Internship. The project covers the full ML pipeline — from raw data collection and 6-month historical backfill, to model training across three algorithms, automated CI/CD retraining, and delivery through an interactive Streamlit dashboard and Flask REST API.

The primary model (Random Forest) achieves 99.66% accuracy. This unusually high figure is explained in Section 12 — it is driven by lag features, not model quality alone. The system has accumulated 328+ automated commits on GitHub over 6 months of continuous operation.

2. Project Overview & Objectives

2.1 Problem Statement

Air pollution is a critical public health issue in Karachi, one of the most populous and polluted cities in South Asia. The city frequently experiences hazardous AQI levels due to vehicle emissions, industrial activity, and seasonal dust. Real-time AQI forecasting enables residents, health authorities, and city planners to make informed decisions.

2.2 Objectives

- Collect real-time hourly pollutant data from OpenWeatherMap Air Pollution API
- Backfill 6 months of historical data (Nov 2025 – May 2026) for model training
- Engineer 13 features including time-based and lag-based variables
- Train and compare Random Forest, Ridge Regression, and TensorFlow Neural Network
- Automate feature and training pipelines via GitHub Actions CI/CD
- Deploy an interactive Streamlit dashboard with 3-day AQI forecasts
- Expose predictions via a Flask REST API (/predict, /forecast, /current)
- Apply SHAP for feature importance analysis and model explainability
- Generate 7 EDA visualizations to identify pollution trends in Karachi

3. System Architecture

The system is structured as three interconnected automated pipelines:

Pipeline	Description	Trigger
Feature Pipeline	Fetches live API data, engineers features, appends to CSV dataset	Every hour — GitHub Actions
Training Pipeline	Loads dataset, trains all 3 models, saves model files to repo	Daily — GitHub Actions
Inference Pipeline	Loads trained models, generates 72-hour AQI forecast for dashboard and API	On-demand (user request)

Data Flow

OpenWeatherMap API → Data Collection Script → Feature Engineering → CSV Feature Store (GitHub) → Model Training → Model Files (.pkl / .keras) → Streamlit Dashboard + Flask API → End User

4. Data Collection & Feature Pipeline

4.1 Data Source

All data was collected from the OpenWeatherMap Air Pollution API (api.openweathermap.org/data/2.5/air_pollution). This API provides hourly pollutant concentrations for Karachi (Lat: 24.8607, Lon: 67.0011). It was chosen because it is free, well-documented, provides historical data up to 1 year back, and delivers all major pollutants in a single endpoint.

4.2 Raw Pollutants Collected

Pollutant	Full Name	Unit
PM2.5	Fine particulate matter	ug/m3
PM10	Coarse particulate matter	ug/m3
NO2	Nitrogen dioxide	ug/m3
CO	Carbon monoxide	ug/m3
O3	Ozone	ug/m3
SO2	Sulphur dioxide	ug/m3
NH3	Ammonia	ug/m3

4.3 Dataset Summary

Parameter	Value
Training Period	November 2025 – May 2026
Total Records	4,410 hourly samples
Collection Frequency	Every hour (automated via GitHub Actions)
Storage Format	CSV — karachi_clean_dataset.csv (GitHub repo)
Missing Values	Forward-fill then dropna()

4.4 Historical Backfill

A backfill script was written to collect 6 months of historical data using the OpenWeatherMap Air Pollution History API. Data was fetched at hourly intervals from November 2025 to May 2026, generating 4,128 historical records. These were merged with live records, deduplicated, and sorted by timestamp to form the final training dataset.

5. Feature Engineering

5.1 Feature Set — 13 Total Features

Feature	Type	Description
pm25	Raw pollutant	PM2.5 concentration
pm10	Raw pollutant	PM10 concentration
no2	Raw pollutant	Nitrogen dioxide
co	Raw pollutant	Carbon monoxide
o3	Raw pollutant	Ozone
so2	Raw pollutant	Sulphur dioxide
nh3	Raw pollutant	Ammonia
hour	Time-based	Hour of day (0–23)
day_of_week	Time-based	Day of week (0=Monday)
month	Time-based	Month of year (1–12)
aqi_lag_1h	Lag feature	AQI value from 1 hour ago
aqi_lag_3h	Lag feature	AQI value from 3 hours ago
aqi_change	Derived	Rate of change: lag_1h minus lag_3h

5.2 Target Variable

The target is the AQI index on the European Air Quality Index 1–5 scale used by OpenWeatherMap: 1=Good, 2=Fair, 3=Moderate, 4=Poor, 5=Very Poor. This is a classification problem with 5 ordered classes.

6. Model Training and Evaluation

6.1 Train/Test Split

An 80/20 chronological split was used. Chronological splitting — rather than random — was chosen to simulate real-world conditions where the model is always tested on future data it has not seen.

6.2 Models Trained

Model 1 — Random Forest (Primary Production Model)

- 100 decision tree ensemble from Scikit-learn
- No feature scaling required — handles non-linear relationships natively
- Compatible with SHAP for explainability
- Saved as: aqi_model.pkl

Model 2 — Ridge Regression (Linear Baseline)

- Regularized linear regression with L2 penalty (alpha=1.0)
- Fast, interpretable baseline to validate RF predictions
- Saved as: aqi_ridge_model.pkl

Model 3 — TensorFlow Neural Network

- Architecture: Dense(64, ReLU) → Dense(32, ReLU) → Dense(1)
- Optimizer: Adam | Loss: MSE | Epochs: 50 | Batch size: 32
- StandardScaler applied to inputs before training
- Saved as: aqi_tf_model.keras

6.3 Model Comparison Results

Metric	Random Forest	Ridge Regression	TensorFlow Neural Net
RMSE	0.0583	0.0009*	0.1700
MAE	0.0034	0.0006*	0.1254
R2 Score	0.9955	1.0000*	0.8699
Accuracy	99.66%	100.0%*	86.99%

* Ridge R2=1.0 and 100% accuracy indicates overfitting on the lag-dominated dataset — see Section 12 for full explanation.

6.4 Why TensorFlow Scores Lower Than RF

The lag features (aqi_lag_1h, aqi_lag_3h) make this a near-trivial task for tree-based models. Random Forest can split perfectly on lag values. Neural networks benefit more from raw sensor data without strongly predictive engineered features. TensorFlow's R2 of 0.87 is still solid performance on unseen data and fulfills the deep learning requirement.

7. CI/CD Automation — GitHub Actions

The entire pipeline runs automatically via GitHub Actions with no manual intervention required after initial setup.

Workflow	Schedule	What It Does
Feature Pipeline	Every hour cron: 0 * * * *	Fetches live pollutant data, engineers features, appends to CSV, commits to GitHub
Training Pipeline	Daily cron: 0 2 * * *	Retrains RF, Ridge, and TF models on latest data, saves model files
CI Check	On every push	Validates code syntax and imports before merging to main branch

As of June 6, 2026 the repository has 328+ automated commits from the GitHub Actions runner, demonstrating over 6 months of continuous pipeline operation.

8. Web Dashboard and Flask API

8.1 Streamlit Dashboard (dashboard.py)

- Real-time AQI display with emoji-coded severity and health advisory text

- Live pollutant metric cards: PM2.5, PM10, NO2, O3
- 3-Day (72-hour) AQI forecast with day-by-day summary cards
- Model selector: switch between Random Forest, Ridge Regression, or TensorFlow Neural Net
- Model comparison panel: side-by-side metrics for all 3 models
- Historical trends tab with Plotly interactive charts
- SHAP explainability tab with feature importance visualization
- Hazardous AQI alert system (triggers at AQI 4 or 5)
- Dark mode toggle and OpenWeatherMap API key input

8.2 Flask REST API (app.py)

Endpoint	Method	Description
/current	GET	Returns live AQI and pollutant levels from OpenWeatherMap
/predict	POST	Accepts pollutant values as JSON, returns AQI class prediction
/forecast	GET	Returns 72-hour AQI forecast as a JSON array

9. SHAP Explainability

SHAP (SHapley Additive exPlanations) was applied to the Random Forest model to explain individual predictions and identify which features drive AQI outcomes most.

Key findings:

- aqi_lag_1h is by far the most influential feature — AQI one hour ago strongly predicts current AQI
- aqi_lag_3h is the second most important, confirming strong temporal autocorrelation in AQI data
- PM2.5 and PM10 are the most impactful raw pollutant features
- Time features (hour, day_of_week) show moderate importance for capturing daily pollution cycles
- NH3 and SO2 show the lowest feature importance in this Karachi dataset

SHAP plots are available interactively in the SHAP Explainability tab of the Streamlit dashboard.

10. EDA Findings

7 visualization plots were generated (stored in eda_plots/):

Plot	Key Finding
AQI Distribution	AQI 3 (Moderate) dominates — 49.7% of all hours. Only 11.7% of hours had good or fair air.
Hourly AQI Trend	Peak AQI during morning rush (8–10 AM) and evening (6–8 PM)
Monthly AQI Trend	Winter months (Nov–Jan) highest AQI due to temperature inversion. May 2026 lowest.
PM2.5 vs AQI Scatter	Strong positive correlation ($r=0.82$) between PM2.5 and AQI

Plot	Key Finding
Correlation Heatmap	Lag features have highest correlation with target AQI (0.97+)
Pollutant Box Plots	PM10 has widest distribution, indicating high day-to-day variability
Day of Week Pattern	Fridays show slightly lower AQI due to reduced traffic in Karachi

11. Deliverables Checklist

#	Requirement	Status	Notes
1	Feature pipeline — fetch raw API data	Done	AQI_Project.py runs hourly via GitHub Actions
2	Time-based and derived features	Done	hour, day_of_week, month, aqi_change
3	Historical data backfill	Done	4,410 records — Nov 2025 to May 2026
4	Feature Store	Partial	GitHub CSV used. Hopsworks/Vertex AI not used — see Section 12
5	Random Forest model	Done	aqi_model.pkl — R2: 0.9955
6	Ridge Regression model	Done	aqi_ridge_model.pkl — linear baseline
7	TensorFlow / Keras model	Done	aqi_tf_model.keras — R2: 0.8699
8	RMSE, MAE, R2 evaluation metrics	Done	All 3 models fully evaluated
9	GitHub Actions — hourly feature pipeline	Done	.github/workflows/ — 328+ automated commits
10	GitHub Actions — daily training pipeline	Done	Retrains all 3 models daily
11	Streamlit dashboard with 3-day forecast	Done	dashboard.py — all tabs functional
12	Flask REST API	Done	app.py — /predict, /forecast, /current
13	EDA with trend analysis	Done	eda.py — 7 plots in eda_plots/
14	SHAP feature importance	Done	Interactive tab in Streamlit dashboard
15	Alerts for hazardous AQI	Done	Alert system triggers at AQI level 4 or 5

12. Honest Limitations & What Was Not Implemented

This section documents the real limitations of the project clearly and honestly. These are not excuses — they are important context for interpreting the results correctly.

12.1 Why the 99.66% Accuracy Is Misleading

The model's 99.66% accuracy is the most important number to explain correctly. At first glance it looks impressive. In reality, it is mostly a result of how the data was structured, not how good the model is at forecasting AQI.

The two lag features — `aqi_lag_1h` (AQI one hour ago) and `aqi_lag_3h` (AQI three hours ago) — are extremely strong predictors of current AQI. This is because AQI in Karachi does not change drastically from hour to hour. If the AQI was 3 one hour ago, it is almost certainly still 3 now. The model has essentially learned: 'predict the same AQI as last hour' and this is correct nearly 99% of the time.

This is a form of data leakage. In a true production deployment where you are predicting 3 days into the future — and lag data from the future is not available — the accuracy would be significantly lower. The model is very good at short-term nowcasting (predicting the next hour) but its 3-day forecast accuracy has not been independently validated without lag features.

This limitation was identified and acknowledged during development. Future work would involve training a separate model without lag features specifically for multi-day forecasting, or using an LSTM architecture that models sequential dependencies differently.

12.2 Hopsworks and Vertex AI Were Not Used

The project requirements specified using Hopsworks or Vertex AI as a managed Feature Store. Neither was used in this project. Here is what was evaluated and why:

- Hopsworks free tier was explored but requires a credit card for account registration, which was not available
- Vertex AI (Google Cloud) similarly requires billing setup even for free-tier usage
- As a practical substitute, the GitHub repository itself was used as a lightweight feature store
- `karachi_clean_dataset.csv` serves as the feature group, version-controlled through Git commits
- This works functionally for a single-city, single-model project but lacks the versioning, lineage tracking, and scalability of a proper managed feature store

This is a genuine gap. The deliverable checklist marks this as Partial rather than Done. If Hopsworks access had been available, the CSV store would have been replaced with a proper feature group with full lineage tracking.

12.3 TensorFlow Was Added Late in the Project

TensorFlow was listed as a required technology. It was not in the original implementation because Random Forest was already achieving very high accuracy and time was limited. A Dense Neural Network ($64 \rightarrow 32 \rightarrow 1$) was added before submission and achieves $R^2=0.87$. This satisfies the requirement to experiment with deep learning, though an LSTM architecture would be more appropriate for this time-series problem and is listed as future work.

12.4 Historical Weather Data Is Missing

The OpenWeatherMap free tier does not provide historical weather data (temperature, humidity, pressure, wind speed). These variables are known to correlate with AQI — for example, temperature inversions trap pollutants near ground level. The model was trained without these variables, which limits its ability to capture

weather-driven pollution events.

12.5 Single City — Not Generalizable

The model was trained exclusively on Karachi data. It will not generalize to other cities without retraining. The lag features in particular are city-specific — the AQI autocorrelation pattern in Karachi may not hold in cities with more volatile pollution patterns.

12.6 Future Work

- Replace CSV feature store with Hopsworks or Vertex AI for proper feature versioning and lineage
- Replace Dense NN with LSTM for genuine sequential time-series modeling
- Train a separate forecasting model without lag features, validated on true 3-day horizons
- Add historical weather data (temperature, humidity, wind) as additional features
- Extend coverage to multiple Pakistani cities — Lahore, Islamabad, Peshawar
- Deploy dashboard to a public URL (Streamlit Cloud) for remote access
- Add SMS or email alert system for hazardous AQI notifications

13. Conclusion

The Pearls AQI Predictor delivers a working, end-to-end machine learning system for air quality forecasting in Karachi. The pipeline covers data collection, feature engineering, three ML models, automated CI/CD, explainability, and a user-facing dashboard — all the core components of a production data science system.

The 99.66% accuracy of the Random Forest model is real but comes with an important caveat: it is largely driven by lag features, not by the model discovering complex patterns. This is clearly documented in Section 12 so that results are interpreted honestly.

Hopsworks and Vertex AI were not used due to billing access issues — this is acknowledged directly. TensorFlow was added and achieves $R^2=0.87$, fulfilling the deep learning requirement. The automated GitHub Actions pipeline has generated 328+ commits over 6 months, demonstrating real sustained operation — not a one-time run.

Overall, the project demonstrates proficiency across the full data science workflow: data engineering, feature engineering, model training and comparison, CI/CD automation, API development, dashboard design, and model explainability.

